CETis 155

MANUAL TÉCNICO

PROYECTO FINAL

Programación 6A
Maestra Teresa Morales Esparza

Presentado por:
Gabriela Negrete ÁLvarez
Ian Anibal Ortega Quiroz
Asael Reyes Belmares
Mauricio Durón Guerrero
Diego Leonardo Maldonado Ramírez

MAIN ACTIVITY: INICIO DE

SECION 1. Declaración de la clase principal

|| public class MainActivity extends AppCompatActivity {
||

Descripción: Esta línea define la clase principal MainActivity, que extiende AppCompatActivity, una clase base para actividades que utilizan las características de la biblioteca de compatibilidad de Android.

2. Declaración de variables

||

```
TextInputEditText editTextEmail, editTextPassword;
Button signIn;
TextView signUp;

FirebaseAuth firebaseAuth = FirebaseAuth.getInstance();
```

||

Descripción: Aquí se declaran las variables que representan los elementos de la interfaz de usuario: campos de texto para el email y la contraseña (editTextEmail, editTextPassword), un botón para iniciar sesión (signIn), un enlace para registrarse (signUp), y una instancia de FirebaseAuth para la autenticación de Firebase.

3. Método onCreate

||

```
@Override protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
editTextEmail = findViewById(R.id.email);
editTextPassword = findViewById(R.id.password); signIn = findViewById(R.id.sign_in); signUp = findViewById(R.id.sign_up);
```

||

Descripción: El método onCreate se llama cuando la actividad es creada. setContentView establece el diseño de la interfaz de usuario desde activity_main.xml. Las variables declaradas anteriormente son vinculadas a los elementos de la interfaz mediante findViewById.

4. Listener para el registro

||

```
signUp.setOnClic
kListener(new
View.OnClickListener
() {
@Overri
de public void onClick(View v) {
Intent intent = new Intent(MainActivity.this, Registro.class);
startActivity(intent);
finish();

}

});
```

||

Descripción: Este bloque establece un OnClickListener para el TextView de registro. Cuando se hace clic en él, se inicia una nueva actividad (Registro.class) y se finaliza la actividad actual (MainActivity).

5. Listener para el inicio de sesión

||

```
signIn.setOnClic
kListener(new
View.OnClickListene
r() {
@Override public void
```

```
onClick(View v) { String email, password; email = String.valueOf(editTextEmail.getTe
xt()); password = String.valueOf(editTextPassword.g
etText());

        if (TextUtils.isEmpty(email)) {

            Toast.makeText(MainActivity.this, "Enter email",
            Toast.LENGTH_SHORT).show(); return;
        }
```

```
        if (TextUtils.isEmpty(password)) {
            Toast.makeText(MainActivity.this, "Enter password", Toast.LENGTH_SHORT).show();
            return;
        }
        firebaseAuth.signInWithEmailAndPassword(email, password)
            .addOnCompleteListener(new OnCompleteListener<AuthResult>() {
                @Override
                public void onComplete(@NonNull Task<AuthResult> task) {
                    if (task.isSuccessful()) {
                        Toast.makeText(MainActivity.this, "Login Successful", Toast.LENGTH_SHORT).show();
                        Intent intent = new Intent(MainActivity.this, Menu_principal.class);
                        startActivity(intent);
                        finish();
                    } else {
                        Toast.makeText(MainActivity.this, "Authentication failed", Toast.LENGTH_SHORT).show();
                    }
                }
            });
}
    });
}
```

||

Descripción: Este bloque establece un OnClickListener para el botón de inicio de sesión. Cuando se hace clic, se obtienen los valores de los campos de email y contraseña. Si alguno de estos campos está vacío, se muestra un mensaje indicando al usuario que debe ingresar los datos. Si ambos campos están completos, se intenta iniciar sesión utilizando FirebaseAuth. Si la autenticación es exitosa, se muestra un mensaje de éxito y se inicia la actividad Menu_principal. Si falla, se muestra un mensaje de error.

REGISTRO: REGISTRAR USUARIO

1. Declaración de la clase principal

```
|| public class Registro extends AppCompatActivity {
||
```

Descripción: Esta línea define la clase Registro, que extiende AppCompatActivity, una clase base para actividades que utilizan las características de la biblioteca de compatibilidad de Android.

2. Declaración de variables

```
||

TextInputEditText editTextEmail, editTextPassword, editTextUsername;
Button signUp;
TextView signIn;

FirebaseAuth firebaseAuth = FirebaseAuth.getInstance();
DatabaseReference databaseReference;
||
```

Descripción: Aquí se declaran las variables que representan los elementos de la interfaz de usuario: campos de texto para el email, la contraseña y el nombre de usuario (editTextEmail, editTextPassword, editTextUsername), un botón para registrarse (signUp), un enlace para iniciar sesión (signIn), una instancia de FirebaseAuth para la autenticación de Firebase, y una referencia a la base de datos de Firebase (databaseReference).

3. Método onCreate

```
||

@Override protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_registro);


editTextEmail = findViewById(R.id.email);
editTextPassword = findViewById(R.id.password);
```

```
editTextUsername = findViewById(R.id.username);
signIn = findViewById(R.id.sign_in); signUp =
findViewById(R.id.sign_up); databaseReference =
FirebaseDatabase.getInstance().getReference().child("
Users");
||
```

Descripción: El método onCreate se llama cuando la actividad es creada. setContentView establece el diseño de la interfaz de usuario desde activity_registro.xml. Las variables declaradas anteriormente son vinculadas a los elementos de la interfaz mediante findViewById. Además, se inicializa la referencia a la base de datos de Firebase apuntando al nodo "Users".

4. Listener para iniciar sesión

```
|| signIn.setOnClickListener(new
View.OnClickListener() {
    @Override public
void onClick(View v) {
        Intent intent = new Intent(Registro.this, MainActivity.class);
        startActivity(intent);
        finish();

    }

  });

||
```

Descripción: Este bloque establece un OnClickListener para el TextView de inicio de sesión. Cuando se hace clic en él, se inicia la actividad MainActivity y se finaliza la actividad actual (Registro).

5. Listener para registrarse

```
|| signUp.setOnClickListener(new
View.OnClickListener() {
    @Override public void onClick(View v)

{ final String email, password, username;
email = String.valueOf(editTextEmail.getText());
password =
String.valueOf(editTextPassword.getText());
```

```
username =
String.valueOf(editTextUsername.getText());


        if (TextUtils.isEmpty(email)) {

            Toast.makeText(Registro.this, "Enter email", Toast.LENGTH_SHORT).show();
            return;
        }

        if (TextUtils.isEmpty(password)){

            Toast.makeText(Registro.this, "Enter password", Toast.LENGTH_SHORT).show();
            return;
        }

        if (TextUtils.isEmpty(username)){

            Toast.makeText(Registro.this, "Enter username", Toast.LENGTH_SHORT).show();
            return;
        }



        firebaseAuth.createUserWithEmailAndPassword(email, password)

            .addOnCompleteListener(new OnCompleteListener<AuthResult>() {
                @Override
                public void onComplete(@NonNull Task<AuthResult> task) {
                    if(task.isSuccessful()){
                        FirebaseUser firebaseUser = firebaseAuth.getCurrentUser();
                        if (firebaseUser != null) {
                            String userId = firebaseUser.getUid();

                            DatabaseReference currentUserDb = databaseReference.child(userId);
currentUserDb.child("username").setValue(username);
                        }

                        Toast.makeText(Registro.this, "Registration successfully",
Toast.LENGTH_SHORT).show(); Intent intent = new Intent(Registro.this,
MainActivity.class);
                        startActivity(intent);
```

```
                    finish();
                } else {
                    Toast.makeText(Registro.this, "Authentication Failed",
Toast.LENGTH_SHORT).show();
                }
            }
        });
}
    });
}
||
```

Descripción: Este bloque establece un OnClickListener para el botón de registro. Cuando se hace clic, se obtienen los valores de los campos de email, contraseña y nombre de usuario. Si alguno de estos campos está vacío, se muestra un mensaje indicando al usuario que debe ingresar los datos. Si todos los campos están completos, se intenta crear un nuevo usuario en Firebase con el email y la contraseña. Si la creación del usuario es exitosa, se obtiene el usuario actual de Firebase, se guarda el nombre de usuario en la base de datos bajo el userId del nuevo usuario, se muestra un mensaje de éxito y se inicia la actividad MainActivity. Si falla, se muestra un mensaje de error.}

TABLAS DE ESTUDIANTES, ENFERMERAS, CITAS Y PSICOLOGAS

En este apartado como en el titulo dice va a ir el código de estas cuatro tablas, en este caso solo vamos a poner el código de una ya que la codificación que se lleva a cabo es la misma, lo único que cambia son las variables o campos a llenar que requiere cada cuestionario.

Cada tabla se compone de tres archivos Java, el principal es donde se llevan a cabo la recopilación de datos, los cambios y las actualizaciones (Estudiantes, Psicólogas, Enfermeras y Citas). Cada uno de estos tiene un adaptador (EstudianteAdapter, PsicologaAdapter, EnfermeraAdapter y CitaAdapter) y un modelo (Estudiante, Psicóloga, Enfermera y Cita).

-Citas

1- Declaracion de variables

||

```
Button btn_add_c, btn_regresar_c, btn_registros_c;

   EditText id_c, fecha_c, hora_c, paciente_c, encargado_c, motivo_c, duracion_c,
observaciones_c;     private FirebaseFirestore mfirestore;
```

||

2- Metodo Oncreate

||

```
   @Override     protected void onCreate(Bundle
savedInstanceState)                                  {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_citas);
String    id    =    getIntent().getStringExtra("id_cita");
mfirestore = FirebaseFirestore.getInstance();


      btn_add_c                                    =
findViewById(R.id.btn_add_cita);         id_c      =
findViewById(R.id.id_citas);         fecha_c       =
findViewById(R.id.fecha_cita);         hora_c      =
findViewById(R.id.hora_cita);         paciente_c =
findViewById(R.id.paciente_cita);
encargado_c                                    =
findViewById(R.id.encargado_cita);
```

```
motivo_c = findViewById(R.id.motivo_cita);
duracion_c = findViewById(R.id.duracion_cita);
observaciones_c =
findViewById(R.id.observaciones_cita);
btn_regresar_c =
findViewById(R.id.btn_regresar_cita);
btn_registros_c =
findViewById(R.id.btn_mostrar_cita);
||
```

Descripción: aquí se declaran las variables a usar para llenar los campos, los botones para ejecutar registro y actualización de datos y a su vez también para redirigir hacia otra ventana

3- Botón para agregar y actualizar con condicionales

```
|| if (id == null || id == ""){
btn_add_c.setOnClickListener(new
View.OnClickListener() {
        @Override

public void onClick(View v) {

            String id__c = id_c.getText().toString().trim();

            String fecha__c = fecha_c.getText().toString().trim();
            String hora__c = hora_c.getText().toString().trim();
            String paciente__c = paciente_c.getText().toString().trim();
            String encargado__c = encargado_c.getText().toString().trim();
            String motivo__c = motivo_c.getText().toString().trim();
            String duracion__c = duracion_c.getText().toString().trim();

            String observaciones__c = observaciones_c.getText().toString().trim();



            if (id__c.isEmpty() && fecha__c.isEmpty() && hora__c.isEmpty() &&

paciente__c.isEmpty() && encargado__c.isEmpty() && motivo__c.isEmpty()
&& duracion__c.isEmpty() &&
                observaciones__c.isEmpty()){
```

```
                Toast.makeText(Citas.this, "Ingresa bien todos los datos",
Toast.LENGTH_SHORT).show();


            } else {

                postCitas(id__c, fecha_c, hora_c, paciente___c, encargado__c, motivo__c,
duracion__c, observaciones_c);

            }

        }

    });
```

En este segmento de código de arriba podemos ver que por medio de condicionales va a determinar si en este caso es nulo o esta vacío y si es ese el caso lo que va a hacer es que solamente va a almacenar los datos ya que es un registro nuevo ejecutando el método “postCitas”.

```
    }else {

        btn_add_c.setText("Update");
        getCitas(id);
        btn_add_c.setOnClickListener(new       View.OnClickListener()       {
          @Override
public void onClick(View v) {

            String id__c = id_c.getText().toString().trim();

            String fecha__c = fecha_c.getText().toString().trim();
            String hora__c = hora_c.getText().toString().trim();
            String paciente__c    =    paciente_c.getText().toString().trim();
            String encargado__c = encargado_c.getText().toString().trim();
            String motivo__c = motivo_c.getText().toString().trim();
            String duracion__c = duracion_c.getText().toString().trim();

            String observaciones__c = observaciones_c.getText().toString().trim();



            if (id__c.isEmpty() && fecha__c.isEmpty() && hora__c.isEmpty() &&

paciente__c.isEmpty() &&                    encargado__c.isEmpty() && motivo__c.isEmpty()
&& duracion__c.isEmpty() &&
                    observaciones__c.isEmpty()){
```

```
                Toast.makeText(Citas.this, "Ingresa bien todos los datos", Toast.LENGTH_SHORT).show();


            } else {

                updateCitas(id__c, fecha__c, hora_c, paciente___c, encargado_c, motivo__c, duracion__c, observaciones_c, id);

            }


        }

    });


}
```

||

En caso contrario a que no se cumpla la condición que anteriormente se declaró ósea que el campo si este lleno y tenga información lo que va a hacer es que va a ejecutar el método “updateCitas” lo que va hacer este método es que va a redirigir al usuario al cuestionario con los campos llenos respecto al registro previamente seleccionado para que pueda modificarlos y al darle al botón update guarde dichas modificaciones.

4- Metodo Update

||

```
private void updateCitas(String id__c, String fecha__c, String hora__c, String paciente__c, String encargado__c, String motivo__c, String duracion__c, String observaciones_c, String id) {
    Map<String, Object> map = new HashMap<>();
    map.put("ID", id_____c);
map.put("Fecha", fecha_____c);
map.put("Hora", hora__c);
map.put("Paciente", paciente___c);
map.put("Encargado", encargado__c);
map.put("Motivo", motivo_____c);
map.put("Duracion", duracion__c);
map.put("Observaciones",
observaciones_c);
```

```
    mfirestore.collection("Citas").document(id).update(map).addOnSuccessListener(new
OnSuccessListener<Void>() {

        @Override          public void
onSuccess(Void unused) {
            Toast.makeText(getApplicationContext(),          "Actualizado          correctamente",
Toast.LENGTH_SHORT).show();

            Intent    intent    =    new    Intent(Citas.this,    Registros_citas.class);
            startActivity(intent);
            finish();



        }

    }).addOnFailureListener(new OnFailureListener() {
        @Override          public void
onFailure(@NonNull Exception e) {

            Toast.makeText(getApplicationContext(),          "Error          al          actualizar",
Toast.LENGTH_SHORT).show();

        }

    });

  }
```

||

Este bloque de código actualiza los detalles de una cita en la base de datos Firestore. Toma los nuevos valores de la cita y los asigna a un objeto HashMap, luego los actualiza en Firestore. Si la actualización es exitosa, muestra un mensaje de éxito y redirige a otra actividad; si falla, muestra un mensaje de error.

5- Método Post

||

```
  private void postCitas(String id__c, String fecha__c, String hora__c,  String  paciente__c,
String encargado_c,

                    String motivo__c, String duracion__c, String observaciones__c)

{       Map<String, Object> map = new HashMap<>();
    map.put("ID", id_c);
map.put("Fecha",  fecha__c);
map.put("Hora", hora__c);
```

```
map.put("Paciente",     paciente____c);
map.put("Encargado", encargado__c);
map.put("Motivo",       motivo_____c);
map.put("Duracion", duracion__c);
map.put("Observaciones",
observaciones_c);
mfirestore.collection("Citas").add(map).add
OnSuccessListener(new
OnSuccessListener<DocumentReference>()
{

        @Override

        public        void        onSuccess(DocumentReference        documentReference)        {
            Toast.makeText(getApplicationContext(), "Registro exitoso",
Toast.LENGTH_SHORT).show();                Intent intent = new Intent(Citas.this,
Registros_citas.class);
            startActivity(intent);




            finish();



        }

      }).addOnFailureListener(new OnFailureListener() {
        @Override            public void
onFailure(@NonNull Exception e) {

            Toast.makeText(getApplicationContext(),                "Registro                denegado",
Toast.LENGTH_SHORT).show();

        }

      });

   }



||
```

Este bloque de código agrega una nueva cita a la base de datos Firestore. Toma los detalles de la cita, los guarda en un objeto HashMap, y luego los agrega a la colección "Citas" en Firestore. Si la operación es exitosa, muestra un mensaje de éxito y redirige a otra actividad; si falla, muestra un mensaje de error.

6- Metodo Get

```
|| private void
getCitas(String id){
    mfirestore.collection("Citas").document(id).get().addOnSuccessListener(new
OnSuccessListener<DocumentSnapshot>() {

        @Override

        public void onSuccess(DocumentSnapshot
documentSnapshot) {          String   idCita   =
documentSnapshot.getString("ID");
            String  fechaCita  =  documentSnapshot.getString("Fecha");
            String horaCita = documentSnapshot.getString("Hora");
            String   pacienteCita   =   documentSnapshot.getString("Paciente");
            String encargadoCita = documentSnapshot.getString("Encargado");
            String motivoCita = documentSnapshot.getString("Motivo");
            String duracionCita = documentSnapshot.getString("Duracion");

            String observacionesCita = documentSnapshot.getString("Observaciones");



            id_c.setText(idCita);
fecha_c.setText(fechaCita);
hora_c.setText(horaCita);
paciente_c.setText(pacienteCita);
encargado_c.setText(encargadoCita)
```

```
motivo_c.setText(motivoCita);
duracion_c.setText(duracionCita);
observaciones_c.setText(observacionesCita);
            }

        }).addOnFailureListener(new OnFailureListener() {
            @Override            public void
 onFailure(@NonNull Exception e) {

                Toast.makeText(getApplicationContext(),            "Error            al            actualizar",
 Toast.LENGTH_SHORT).show();



            }

        });



    }

 ||
```

Este bloque de código obtiene los detalles de una cita específica desde la base de datos Firestore. Toma el ID de la cita como entrada, realiza una consulta en Firestore para obtener los detalles correspondientes y luego los muestra en la interfaz de usuario para su visualización o edición. Si hay algún error durante el proceso, muestra un mensaje de error.

Y solo como recordatorio tomamos el ejemplo la tabla citas porque era la más corta pero como ya repetimos todas las tablas llevan los mimos métodos, misma funcionalidad solo con diferentes nombres y variables depende la tabla.

-Cita

```
|| public class
Cita                {
```

```
String ID, Fecha, Hora, Paciente, Encargado, Motivo, Duracion, Observaciones;
public Cita(){}

    public Cita(String ID, String fecha, String hora, String paciente, String encargado, String
motivo, String duracion, String observaciones) {

        this.ID = ID;

       Fecha = fecha;

       Hora = hora;

        Paciente     =     paciente;
        Encargado  =  encargado;
        Motivo       =       motivo;
        Duracion = duracion;
        Observaciones = observaciones;

    }



    public   String   getID()   {
return ID;
    }



    public   void   setID(String   ID)   {
        this.ID = ID;
    }



    public    String    getFecha()    {
return Fecha;
    }



    public   void   setFecha(String   fecha)   {
        Fecha = fecha;
    }
```

```
   public String getHora() {
return Hora;
   }


   public  void  setHora(String  hora)  {
     Hora = hora;
   }


   public   String   getPaciente()   {
return Paciente;
   }


   public   void   setPaciente(String   paciente)   {
      Paciente = paciente;
   }


   public   String   getEncargado()   {
return Encargado;
   }


   public   void   setEncargado(String   encargado)   {
      Encargado = encargado;
   }


   public   String   getMotivo()   {
return Motivo;
   }


   public  void  setMotivo(String  motivo)  {
      Motivo = motivo;
   }
```

```
   public   String   getDuracion()   {
return Duracion;
   }


   public   void   setDuracion(String   duracion)   {
      Duracion = duracion;
   }



   public  String  getObservaciones()  {
return Observaciones;
   }



   public   void   setObservaciones(String   observaciones)   {
      Observaciones = observaciones;
   }

}
```

||

Este bloque de código define una clase llamada Cita, que sirve como modelo para representar los detalles de una cita

Atributos: La clase tiene atributos para almacenar los detalles de una cita: ID, Fecha, Hora, Paciente, Encargado, Motivo, Duración y Observaciones.

Constructores: La clase tiene dos constructores. Uno vacío y otro que inicializa todos los atributos de la cita.

Métodos getters y setters: Se proporcionan métodos para acceder y modificar los valores de los atributos de la cita.

Un requisito muy importante es que en las variables de tipo String se tiene que llamar exactamente iguales a como están escritas en la base de datos.

-CitaAdapter

||

// Declaración de la clase CitaAdapter public class CitaAdapter extends FirestoreRecyclerAdapter<Cita, CitaAdapter.ViewHolder> { private FirebaseFirestore mFirestore = FirebaseFirestore.getInstance(); Activity activity;

// Constructor de CitaAdapter public CitaAdapter(@NonNull FirestoreRecyclerOptions<Cita> options, Activity activity) {
super(options);
this.activity = activity;
}

||

Este bloque de código define la clase CitaAdapter, que extiende FirestoreRecyclerAdapter. El adaptador se utiliza para trabajar con datos de Firestore en un RecyclerView. El constructor recibe las opciones de configuración del adaptador y una referencia a la actividad actual.

||

// Método onBindViewHolder para asignar datos a los elementos de la vista de un ítem del RecyclerView

@Override protected void onBindViewHolder(@NonNull ViewHolder viewHolder, int i, @NonNull Cita Cita) {
DocumentSnapshot documentSnapshot = getSnapshots().getSnapshot(viewHolder.getAbsoluteAdapterPositi

on()); final String id = documentSnapshot.getId();

// Asignación de datos a los elementos de la vista viewHolder.id_c.setText("ID: "+Cita.getID()); viewHolder.fecha_c.setText("Fecha de la cita: "+Cita.getFecha()); viewHolder.hora_c.setText("Hora de la cita: "+Cita.getFecha()); viewHolder.paciente_c.setText("Nombre del paciente: "+Cita.getPaciente()); viewHolder.encargado_c.setText("Encargado

```
de la cita: "+Cita.getEncargado());
viewHolder.motivo_c.setText("Motivo: "+Cita.getMotivo());
viewHolder.duracion_c.setText("Duracion de la consulta: "+Cita.getDuracion());
viewHolder.observaciones_c.setText("Observaciones: "+Cita.getObservaciones());


  // Configuración de OnClickListener para los botones de eliminar y editar
viewHolder.btn_eliminar.setOnClickListener(new View.OnClickListener() {
      @Override public void onClick(View v) {
          eliminarCita(id);

      }

  });



  viewHolder.btn_editar.setOnClickListener(new View.OnClickListener() {
      @Override public void onClick(View v) {

          Intent i = new Intent(activity, Citas.class);
          i.putExtra("id_cita", id);
activity.startActivity(i);

      }

  });

}
```

||

El método onBindViewHolder asigna los datos de una cita a los elementos de la vista de un ítem del RecyclerView. Configura también OnClickListeners para los botones de eliminar y editar citas.

||

```
// Método para eliminar una cita de Firestore
private void eliminarCita(String id) {
   mFirestore.collection("Citas").document(id).delete().addOnSuccessListener(new
OnSuccessListener<Void>() {

      @Override        public void
onSuccess(Void unused) {
         Toast.makeText(activity, "Eliminado exitosamente", Toast.LENGTH_SHORT).show();

      }

   }).addOnFailureListener(new OnFailureListener() {
      @Override        public void
onFailure(@NonNull Exception e) {

         Toast.makeText(activity, "Error al eliminar", Toast.LENGTH_SHORT).show();

      }

   });

}
```

||

El método eliminarCita elimina una cita de Firestore utilizando el ID proporcionado. Accede a la colección "Citas" en Firestore y elimina el documento correspondiente al ID. Luego, muestra un mensaje indicando si la eliminación fue exitosa o si ocurrió un error.

||

// Método onCreateViewHolder para inflar el diseño de la vista de un ítem del RecyclerView

```
@NonNull @Override public ViewHolder onCreateViewHolder(@NonNull
ViewGroup parent, int viewType) {
   View v = LayoutInflater.from(parent.getContext()).inflate(R.layout.view_cita_single, parent,
false);     return new ViewHolder(v);
}
```

||

El método onCreateViewHolder infla el diseño de la vista de un ítem del RecyclerView a partir de un archivo de diseño XML y devuelve una nueva instancia del ViewHolder que contiene la vista inflada.

||

```
// Clase interna ViewHolder que contiene referencias a los elementos de la vista de
un ítem del RecyclerView public class ViewHolder extends
RecyclerView.ViewHolder {
   TextView id_c, fecha_c, hora_c, paciente_c, encargado_c, motivo_c, duracion_c,
observaciones_c;     Button btn_eliminar, btn_editar;

   public ViewHolder(@NonNull View itemView) {
      super(itemView);

      // Referencias a los elementos de la vista
btn_eliminar =
itemView.findViewById(R.id.btn_eliminar_cita);
btn_editar = itemView.findViewById(R.id.btn_editar_cita);
id_c = itemView.findViewById(R.id.id_citas);        fecha_c =
itemView.findViewById(R.id.fecha_cita);        hora_c =
itemView.findViewById(R.id.hora_cita);        paciente_c =
itemView.findViewById(R.id.paciente_cita);
encargado_c =
itemView.findViewById(R.id.encargado_cita);
motivo_c = itemView.findViewById(R.id.motivo_cita);
```

```
duracion_c = itemView.findViewById(R.id.duracion_cita);
observaciones_c =
itemView.findViewById(R.id.observaciones_cita);
    }

}
```

||

La clase interna ViewHolder contiene referencias a los elementos de la vista de un ítem del RecyclerView, como TextViews y botones. En su constructor, inicializa estas referencias utilizando findViewById.

REGISTROS CITA

En esta clase es el cascaron donde se van a presentar todos los datos que estén almacenado en la base de datos, le damos un orden y establecemos las conexiones necesarias para poder mostrar los datos

```
|| public class Registros_citas extends
AppCompatActivity {

   Button btn_regresar;
   RecyclerView mRecycler;
   CitaAdapter mAdapter;
   FirebaseFirestore mFirestore;

   @Override protected void onCreate(Bundle
savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_registros_citas
);



      mFirestore = FirebaseFirestore.getInstance();
mRecycler = findViewById(R.id.recyclerViewSingle_citas);
mRecycler.setLayoutManager(new
LinearLayoutManager(this)); Query query =
mFirestore.collection("Citas");

      FirestoreRecyclerOptions<Cita> firestoreRecyclerOptions = new
FirestoreRecyclerOptions.Builder<Cita>().setQuery(query, Cita.class).build();

      mAdapter = new CitaAdapter(firestoreRecyclerOptions,
this); mAdapter.notifyDataSetChanged();
mRecycler.setAdapter(mAdapter);
```

```
        btn_regresar = findViewById(R.id.btn_regresar_cit);




        btn_regresar.setOnClickListener(new View.OnClickListener() {
            @Override
public void onClick(View v) {
                Intent intent = new Intent(Registros_citas.this, Citas.class);
                startActivity(intent);
                finish();



            }
        });



    }



    @Override protected
void onStart() {
super.onStart();
mAdapter.startListening();
    }



    @Override protected
void onStop() {
super.onStop();
mAdapter.stopListening();
    }
}
```

||

Este bloque de código define la clase Registros_citas, que extiende AppCompatActivity y muestra una lista de citas almacenadas en Firestore.

Clase Registros_citas:Se encarga de mostrar los registros de citas almacenados en Firestore.

En el método onCreate, se inicializa la instancia de Firestore, se configura el RecyclerView y se crea un adaptador (CitaAdapter) para mostrar los datos.

El botón btn_regresar permite regresar a la actividad principal (Citas).

En el método onStart, se inicia la escucha del adaptador para actualizar los datos cuando la actividad se muestra.

En el método onStop, se detiene la escucha del adaptador cuando la actividad deja de mostrarse.